Group members

 Ahammad, Jony

Kumara, Muditha

 Podkorytov, Nikolai

Shrestha Bata, Prabesh

Your answer NOT SUBMITTED

Assignment

Part 1: Exploring and Applying Ensemble Methods

📌 Objective

The objective of this assignment is twofold:

1. **Study and Analyze** the provided **alpha-phase** ensemble learning program, identifying potential issues and understanding its workflow.
2. **Apply Ensemble Methods** (Bagging, Boosting, Random Forest, XGBoost, etc.) on your **previous assignment problems and datasets**, compare the results, and reflect on the improvements.

🔍 Phase 1: Understanding the Alpha-Phase Program

Step 1: Running the Program

- Download the **Python program** provided (you may also receive it as a `.py` file).
- Ensure you have the necessary **dependencies installed**:
  `pip install numpy pandas scikit-learn matplotlib xgboost`
- Run the program.
- Explore the **Graphical User Interface (GUI)** and interact with different functionalities.

Step 2: Identifying Issues

- Since the program is in its **alpha phase**, it may contain **bugs** or **incomplete features**.
- Identify any **errors, usability issues, or improvements** needed.
- Pay special attention to:
    - **Dataset loading**
    - **Hyperparameter tuning**
    - **Visualization outputs**
    - **Performance comparisons between models**
- **Document** your findings, including screenshots if needed.

Step 3: Understanding the Implementation

- Study the **EnsembleDemoCore** class to understand how models are being trained.
- Check how **classification, regression, clustering, anomaly detection, and dimensionality reduction** are handled.
- Explain how hyperparameters affect **ensemble methods** in different tasks.

📊 Phase 2: Applying Ensemble Methods to Your Previous Work

Step 4: Implementing Ensemble Methods on Your Own Dataset

- Take your **previous assignment dataset** (from regression, classification, or clustering tasks).
- Apply **at least two ensemble methods**:
    - **For Classification**: Random Forest, XGBoost, AdaBoost.
    - **For Regression**: Random Forest Regressor, XGBoost Regressor, AdaBoost Regressor.
    - **For Clustering**: K-Means, DBSCAN, Agglomerative Clustering.
    - **For Anomaly Detection**: Isolation Forest, Local Outlier Factor.
- **Tune the hyperparameters** using `GridSearchCV` or `RandomizedSearchCV`.

Step 5: Evaluating Performance

- Compare your previous model's performance against ensemble methods.
- Use appropriate **evaluation metrics**:
    - Classification: Accuracy, F1-score, ROC-AUC.
    - Regression: RMSE, R^2.
    - Clustering: Silhouette Score, Adjusted Rand Index.
- **Visualize** the results using:
  `import matplotlib.pyplot as plt`
- Provide a **side-by-side comparison table** of model performance.

Step 6: Reflection

- Answer the following questions in your report:
    1. **What were the key differences** between your previous model and ensemble methods?
    2. **Which ensemble method performed the best** for your dataset? Why?
    3. **What hyperparameters had the most significant impact** on the model's performance?
    4. **Did you identify any major issues in the alpha-phase program? How could they be improved?**

📌 Submission Guidelines

- Submit a **well-structured report** (PDF format) covering:
    - **Findings from the alpha-phase program**.
    - **Comparison of different ensemble methods** on your dataset.
    - **Performance metrics and visualizations**.
    - **Reflections on model improvements**.
- Include your **Python code** snippets and screenshots.

🔗 References

- Scikit-Learn Ensembles:https://scikit-learn.org/stable/modules/ensemble.html
- XGBoost Documentation: https://xgboost.readthedocs.io/en/latest/
- Bagging vs. Boosting: https://towardsdatascience.com/bagging-vs-boosting-9c91cd29c79f

---

💡 Notes

- This part requires **both theoretical understanding and practical application**.
- Debugging and improving the alpha-phase program will give you **valuable experience in working with real-world machine learning projects**.

Part 2: Evaluating and Validating Your Model

In this assignment, you will work in your groups to evaluate the validity of your previously trained models using the concepts covered in the session. Your task is to assess the performance and generalization of your models by implementing appropriate validation techniques and calculating key evaluation metrics.

Tasks:

1. **Apply Cross-Validation Techniques:**
    - If you used a simple train/test split before, implement **K-Fold Cross-Validation** or **Stratified K-Fold** (if your dataset is imbalanced: check for it!).
    - If working with sequential data (e.g., time-series), use **TimeSeriesSplit** or **rolling/expanding window validation**.
    - Compare the performance metrics obtained across different folds.
2. **Compute Classification or Regression Metrics:**
    - For **classification models**, calculate the following:
        - **Confusion Matrix** (identify TP, TN, FP, FN)
        - **Accuracy** (only if the dataset is balanced)

    - **Precision, Recall, and F1-score** (especially for imbalanced datasets)
    - **ROC-AUC or PR-AUC** (for evaluating probabilistic models)
  - For **regression models**, compute:
    - **Mean Squared Error (MSE) and Mean Absolute Error (MAE)**
    - **R-Squared and Adjusted R-Squared**
    - **Root Mean Squared Error (RMSE)**
  - Visualize the differences in performance using graphs and tables.
3. **Analyze and Compare Results:**
   - Interpret the metrics: What do they reveal about your model's strengths and weaknesses?
   - Compare results from different validation techniques: Do the metrics remain stable across different folds?
   - Identify any signs of **overfitting or underfitting**.
4. **Document Findings and Observations:**
   - Summarize key observations from your evaluation.
   - Include any relevant visualizations (e.g., confusion matrix, ROC curve, error distributions).
   - Reflect on how the choice of validation technique and metric affects the interpretation of model performance.

Submission Requirements:

- A **Jupyter Notebook** or Python script with all calculations, visualizations, and evaluation metrics.
- A short **group report** (1–2 pages) summarizing:
  - The validation techniques applied.
  - The key metrics calculated and their interpretations.
  - Insights gained about the model's performance.

- +Screenshot and corresponding explanations.

Part 3: Incorporating Advanced Scikit-Learn Techniques

In this final part, you will elevate your machine learning workflows by applying advanced features of Scikit-Learn covered in the recent session (see the provided slides on "Advanced Scikit-Learn").

---

1. Building Pipelines for Your Data

- **Objective**: Streamline your preprocessing steps (such as handling missing values, scaling, or encoding) alongside your chosen model(s) into a cohesive pipeline.
- **Tasks**:

    1. Create a **Pipeline** that includes any transformations you applied manually before (e.g., imputation, scaling).
    2. Ensure you fit the pipeline **only on your training data** to avoid data leakage (consult the "\texttt{Pipeline}" slides for examples).
    3. Compare your pipeline results to your previous approach. Did data leakage issues appear before? Are your new metrics more reliable?

---

2. Feature Engineering & Scaling

- **Objective**: Improve model performance and interpretability by engineering features and applying the right scaling methods.
- **Tasks**:

    1. **Feature Creation**: Try at least one method (e.g., polynomial features, log transforms). Evaluate if it helps or hinders performance.
    2. **Categorical Encoding**: Use \texttt{OneHotEncoder}, \texttt{OrdinalEncoder}, or another approach if relevant for your

dataset.
3. **Scaling**: Select an appropriate scaler (\texttt{StandardScaler}, \texttt{MinMaxScaler}, or \texttt{RobustScaler}) based on your data's outliers or distribution.
4. Document any changes in performance due to these transformations (reference the "Feature Engineering & Scaling" slides).

---

3. Model Persistence (Saving & Loading)

- **Objective**: Create a production-ready workflow by saving your final pipeline (including all transformations) to be reloaded without retraining.
- **Tasks**:

    1. After training your pipeline on the final dataset split, **save** it using \texttt{joblib.dump()}.
    2. **Load** the saved pipeline (\texttt{joblib.load()}) in a new script or notebook. Confirm it can still produce the same predictions (reference the "Model Persistence (Saving & Loading)" slides).
    3. Reflect on how this approach helps if you were to deploy your model or share it with teammates.

---

4. Assignment Deliverables

1. **Notebook/Script**:

    - Show your pipeline construction, feature engineering, scaling steps, and model persistence code snippets.
    - Compare metrics **before** and **after** these advanced transformations or techniques.

2. **Group Report** (brief):

    - **Pipelines**: What transformations were added, and how did they improve or simplify your workflow?

- **Feature Engineering & Scaling**: Which methods did you try? Did polynomial features or log transforms help? Did robust scaling handle outliers better?
- **Model Persistence**: Confirm you can load the saved model/pipeline and generate consistent predictions.
- **Insights & Challenges**: Summarize how these advanced scikit-learn features impacted your ensemble methods' performance, maintainability, and reproducibility.

---

5. Tips & Reminders

- **Check for Data Leakage**: Ensure transformations are fit on training data only—never on the entire dataset.
- **Keep it Modular**: If you have multiple transformations, chaining them in a pipeline avoids missing or inconsistent steps.
- **Version Control**: Note the scikit-learn and Python versions in your environment to ensure reproducibility.
- **Links for Reference**:
  - Scikit-Learn Pipelines Documentation
  - Feature Engineering Guide
  - Model Persistence